

Special Participation E: Equation Intuition Annotator

I used Claude's Artifacts to vibe code an equation annotator to help gain intuition for how often monstrous equations work. I think you need a Claude account to use the actual artifact, but you can replicate this by feeding the same prompt to a chatbot instead of using my tool to send an API request.

Here is the prompt if you want to use an image of an equation:

None

I've uploaded an image containing a deep learning equation. Please identify the equation and provide a breakdown with these sections:

1. WHAT IT IS: One sentence describing what this equation represents
2. SYMBOL BREAKDOWN: Explain each symbol/term and what it represents intuitively
3. WHY IT WORKS: Explain the intuition behind why this formulation makes sense
4. WHAT IF WE CHANGE THINGS: For each major variable/parameter, explain what happens when you:
 - Increase it
 - Decrease it
 - Set it to zero (if applicable)
 - Remove it entirely
5. COMMON MISCONCEPTIONS: What do students often get wrong about this?
6. VISUAL INTUITION: Describe a geometric or visual way to think about this

Use LaTeX formatting for any mathematical expressions (use $...$ for inline math and
$$...$$
 for display math).

Be concise, intuitive, and focus on building understanding rather than just stating facts.

Here is the prompt if you want to type an equation:

None

I've uploaded an image containing a deep learning equation. Please identify the equation and provide a breakdown with these sections:

1. WHAT IT IS: One sentence describing what this equation represents
2. SYMBOL BREAKDOWN: Explain each symbol/term and what it represents intuitively
3. WHY IT WORKS: Explain the intuition behind why this formulation makes sense

4. WHAT IF WE CHANGE THINGS: For each major variable/parameter, explain what happens when you:

- Increase it
- Decrease it
- Set it to zero (if applicable)
- Remove it entirely

5. COMMON MISCONCEPTIONS: What do students often get wrong about this?

6. VISUAL INTUITION: Describe a geometric or visual way to think about this

Use LaTeX formatting for any mathematical expressions (use \$ for inline math and
$$for display math).$$

Be concise, intuitive, and focus on building understanding rather than just stating facts.

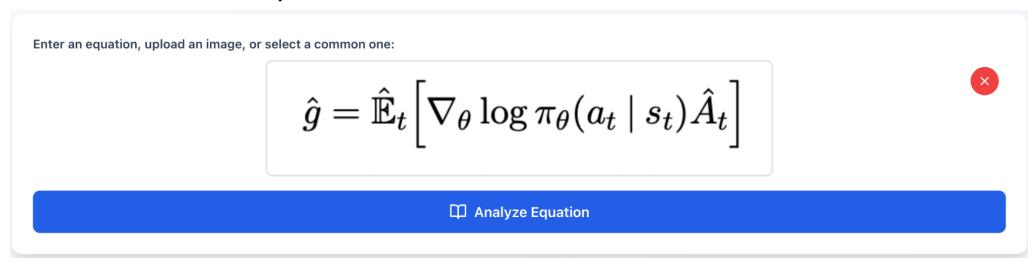
The link is here

<https://claude.ai/public/artifacts/4a9d7ab1-dbcc-44fc-b0e7-3afe1e45f38d>

Here I demonstrate its use on some equations in the PPO paper.

Gradient Estimator

I want to learn about the policy gradient estimator and why it works, so I provide this screenshot of the equation.



Here it gives me a quick summary. Nothing too special, but I would rewrite this to try to connect this better to specific resources and provide citations (e.g. lectures, course homeworks, papers online, etc).

Equation from image

1. WHAT IT IS

This is the **policy gradient equation** from reinforcement learning, specifically the expected gradient of the log probability of actions weighted by advantages, used to update policy parameters in algorithms like REINFORCE and Actor-Critic methods.

Here the variables are identified and considered on their own. I believe that these are correct.

2. SYMBOL BREAKDOWN

$\hat{g}$: Policy gradient estimator (output) - the estimated gradient used to update policy parameters during training

$\mathbb{E}_t$: Expectation operator over time steps (mathematical operator) - averages over all possible trajectories or time steps in an episode

∇_{θ} : Gradient operator with respect to θ (mathematical operator) - computes partial derivatives of the following expression with respect to each policy parameter

$\log$: Natural logarithm function (mathematical operator) - ensures the gradient has nice mathematical properties and relates to the score function

$\pi_{\theta}(a_t|s_t)$: Policy probability distribution (learned function) - gives probability of taking action a_t in state s_t under policy parameterized by θ , values in $[0,1]$ with all actions summing to 1

a_t : Action at time step t (input/random variable) - the specific action taken by the agent, can be discrete (integers) or continuous (real numbers)

s_t : State at time step t (input/environment variable) - the current observation/state of the environment, domain-specific (could be pixel values, coordinates, etc.)

$\hat{A}_t$: Advantage estimate at time step t (computed value) - measures how much better action a_t was compared to the average action in state s_t , typically ranges from $-\infty$ to $+\infty$ but often normalized

θ : Policy parameters (learned parameters) - the weights/parameters of the neural network or function approximator representing the policy, updated via gradient ascent

Here is the useful part that gives us the intuition:

3. WHY IT WORKS

The intuition is beautifully simple: we want to increase the probability of actions that led to better-than-expected outcomes and decrease the probability of actions that led to worse-than-expected outcomes. The gradient $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$ tells us the direction to change θ to increase the probability of action a_t . When we multiply by a positive advantage $\hat{A}_t$, we move in that direction (reinforcing good actions). When $\hat{A}_t$ is negative, we move in the opposite direction (discouraging bad actions). The expectation ensures we consider all possible trajectories.

4. WHAT IF WE CHANGE THINGS

$\hat{A}_t$ (Advantage)

- **Increase it:** Policy becomes more confident in action a_t ; if $\hat{A}_t = 10$ vs $\hat{A}_t = 1$, the gradient is 10x larger, causing more aggressive updates toward selecting a_t
- **Decrease it:** Smaller updates; as $\hat{A}_t \rightarrow 0$, the gradient approaches zero and learning stops for that action
- **Set to zero:** $\hat{g} = 0$, no learning occurs - the policy doesn't update regardless of actions taken
- **Negative values:** Policy learns to avoid action a_t ; if $\hat{A}_t = -5$, we get $5 \times$ the gradient in the direction that decreases $\pi(a_t|s_t)$
- **Typical values:** Often normalized to have mean 0 and std 1, so roughly $[-3, 3]$ range

$\pi_{\theta}(a_t|s_t)$ (Action Probability)

- **Increase it:** $\log \pi_{\theta}(a_t|s_t)$ approaches 0, gradient magnitude decreases; if $\pi = 0.9$, $\log(0.9) = -0.105$
- **Decrease it:** Gradient magnitude increases dramatically; if $\pi = 0.01$, $\log(0.01) = -4.605$
- **Set to zero:** $\log(0) = -\infty$

The most striking issue is the improper formatting and incomplete completion. I think that this is due to the use of Claude API vs the chatbot interface. There is probably a token limit for using the artifacts for free online. I'm not so sure about the broken LaTeX, but I rarely see this happen using the chat interface.

One issue I don't like is that it doesn't clarify that advantage is a function of s and a , but the equation in the picture doesn't do this either, and this is somewhat of a stylistic choice. Other than that, I think this looks good and is helpful. I would probably adjust the prompt to add a few more clarifying sentences for each variable at some point later though.

KL Penalty

Enter an equation, upload an image, or select a common one:

$$L^{KL PEN}(\theta) = \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

Analyze Equation

This equation is a little more complex, so let's see what happens.

AI Explanation

Equation from image

WHAT IT IS

This is the KLPEN (KL-divergence Penalty) loss function used in reinforcement learning from human feedback (RLHF), specifically in training language models to align with human preferences while preventing the model from deviating too far from a reference policy.

SYMBOL BREAKDOWN

$L^{KL PEN}(\theta)$: KLPEN loss function (output) - the total loss we minimize during training, combining reward maximization with KL penalty

$\mathbb{E}_t$: Expectation operator (mathematical operator) - takes the expected value over all time steps t in the trajectory

t : Time step index (input) - discrete time points in a sequence, typically ranges from 0 to sequence length

$\pi_{\theta}(a_t | s_t)$: Current policy (learned parameter) - probability distribution over actions given state, parameterized by θ , values in $[0, 1]$ and sum to 1

$\pi_{\theta_{old}}(a_t | s_t)$: Reference/old policy (fixed parameter) - frozen probability distribution from previous training iteration, serves as anchor point

a_t : Action at time t (input) - the specific action taken, for language models this is typically a token ID

s_t : State at time t (input) - the current context/state, for language models this is the sequence of previous tokens

$\hat{A}_t$: Advantage estimate (computed value) - measures how much better action a_t is compared to average, can be positive or negative

β : KL penalty coefficient (hyperparameter) - controls strength of regularization, typically ranges from 0.01 to 1.0

$\text{KL}[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]$: KL divergence (computed value) - measures difference between old and new policies, always non-negative

The quick summary seems good. One thing I would do to modify the prompt though is ask it to be more verbose about the symbol definitions.

WHY IT WORKS

The equation balances two competing objectives: maximizing rewards (first term) while staying close to a reference policy (second term). The ratio $\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$ measures how much the new policy differs from the old one for specific actions. Multiplying by the advantage $\hat{A}_t$ means we increase probability of good actions and decrease probability of bad actions. The KL penalty prevents catastrophic forgetting and maintains some similarity to the original model behavior.

WHAT IF WE CHANGE THINGS

β (KL penalty coefficient):

- **Increase** ($\beta \rightarrow 1.0$): Model changes very conservatively, stays very close to reference policy. Example: $\beta = 1.0$ might keep policy changes under 1% per update.
- **Decrease** ($\beta \rightarrow 0.1$): Model can deviate more from reference policy, faster learning but risk of instability. Example: $\beta = 0.01$ allows 10x more deviation.
- **Set to zero** ($\beta = 0$): $\mathbb{E}[\frac{p(\theta(a_t | s_t))}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t]$ - becomes pure policy gradient, no regularization, can lead to catastrophic policy collapse.
- **Remove entirely**: Impossible - this becomes a different algorithm entirely.
- **Typical values**: 0.01 to 0.2, often started high (0.1) and decayed during training.

$\hat{A}_t$ (Advantage):

- **Increase**: Stronger learning signal, but if consistently high might indicate poor baseline estimation.
- **Decrease toward zero**: Weaker learning signal, policy changes become minimal.
- **Set to zero**: No learning from first term, only KL penalty remains, policy frozen at reference.
- **Remove entirely**: No reward signal, only regularization - model won't improve.
- **Typical values**: Normalized to have zero mean, standard deviation around 1.0.

Contentwise, I think that this is correct. I would again want this to be more verbose. The LaTeX is broken again but this generally isn't an issue in chat.

The rest is cut off and I think it's because I hit the token limit. For ideal results I would run the prompts I provide in a chatbot instead, but I do like the website format.